

BAŞLIK: API DOKÜMANTASYONU VE TEST ETME

TÜR: Uygulamalı/Teknik Detaylı Anlatım

BÖLÜM 1: NEDEN DOKÜMANTASYON KRİTİKTİR?

Bir API, kodun kendisi kadar iyi olsa bile, ONU NASIL KULLANACAĞINI kimse bilmiyorsa değersizdir. 22.1'de gördüğümüz "API ekonomisi" örneklerinin (Stripe, Twilio) hepsinin ortak noktası, son derece kaliteli dokümantasyona sahip olmalarıdır — bir geliştiricinin bir API'yi seçme kararında dokümantasyon kalitesi çoğu zaman teknik özelliklerden bile daha belirleyicidir.

1.1. OpenAPI Spesifikasyonu (eski adıyla Swagger)

OpenAPI, bir REST API'nin TÜM uç noktalarını, parametrelerini, istek/yanıt şemalarını ve olası hata kodlarını, makine tarafından okunabilir bir JSON/YAML formatında tanımlayan endüstri standardıdır. Bu "tek doğruluk kaynağı" (single source of truth) üzerinden: (1) İnteraktif dokümantasyon sayfaları otomatik üretilebilir, (2) İstemci kütüphaneleri (SDK) farklı dillerde otomatik kod üretimiyle oluşturulabilir, (3) Sözleşme testleri (contract testing, bkz. Bölüm 4) otomatikleştirilebilir.

1.2. FastAPI'de Otomatik Dokümantasyon

22.6'da inşa ettiğimiz FastAPI uygulaması, hiçbir ek kod yazmadan, sadece tanımladığımız path operations ve Pydantic modelleri üzerinden, OpenAPI şemasını OTOMATİK üretir ve bunu iki farklı arayüzle sunar: /docs (Swagger UI — her uç noktayı doğrudan tarayıcı üzerinden "deneyebileceğiniz" interaktif bir arayüz) ve /redoc (ReDoc — daha okuma odaklı, üç panelli statik bir dokümantasyon görünümü). Bu, FastAPI'yi seçmemizin (bkz. 22.6.1, Bölüm 1) en somut faydalarından biridir: dokümantasyon asla kodla "senkron dışı" (out of sync) kalmaz, çünkü doğrudan koddan üretilir.

1.3. Docstring'ler ve Örneklerin Önemi

Otomatik üretilen şema teknik olarak DOĞRUDUR ama tek başına her zaman YETERİNCE AÇIKLAYICI değildir. Her path operation fonksiyonuna eklenen docstring'ler FastAPI tarafından otomatik olarak Swagger UI'a "Description" alanı olarak yansıtılır; Pydantic modellerindeki Field(..., example="Suç ve Ceza") gibi örnek değerler de dokümantasyonda gerçekçi örnekler olarak görünür. İyi dokümantasyon sadece "ne" değil "neden" ve "ne zaman" sorularına da cevap verir (örn. "bu uç nokta neden 429 dönebilir, ne zaman kullanılmalı").

BÖLÜM 2: MANUEL TEST ARAÇLARI — POSTMAN VE INSOMNIA

Bir API'yi geliştirirken, her değişiklikten sonra tarayıcıdan (sadece GET için çalışır) ya da elle Python kodu yazarak test etmek verimsizdir. Postman ve Insomnia gibi araçlar, grafik arayüzden istek oluşturmayı, kaydetmeyi, koleksiyonlar halinde organize etmeyi ve ORTAM DEĞİŞKENLERİ (environment variables — örn. "BASE_URL"ı geliştirme/test/production arasında tek tıkla değiştirmek) tanımlamayı sağlar. Bir Postman koleksiyonu, aynı zamanda ekip arkadaşlarıyla paylaşılabilir, "bu API'nin nasıl kullanılacağını" canlı bir örneği haline gelir — genellikle OpenAPI şemasından otomatik olarak İÇE AKTARILABİLİR (import).

BÖLÜM 3: OTOMATİK TEST TÜRLERİ

Manuel test, geliştirme sırasında hızlı geri bildirim için değerlidir ama bir API büyüdükçe, HER değişiklikten sonra TÜM uç noktaları elle test etmek imkansız hale gelir. Bu yüzden production API'leri otomatik testlerle korunur; bu testler birkaç seviyede sınıflandırılır:

3.1. Birim Testleri (Unit Tests)

İş mantığının (business logic) küçük, izole parçalarını (örn. "bir kitabın ödünç verilebilir olup olmadığını kontrol eden fonksiyon") test eder — HTTP katmanına, veritabanına hiç dokunmadan. En hızlı çalışan test türüdür ve genellikle en çok sayıda bu türden test yazılır.

3.2. Entegrasyon Testleri (Integration Tests)

Birden fazla bileşenin BİRLİKTE doğru çalıştığını doğrular — örneğin "bir POST isteği gerçekten veritabanına doğru satırı yazıyor mu?". 22.6.2 projemizdeki pytest testleri, FastAPI'nin TestClient'ı ve test amaçlı ayrı bir SQLite veritabanı kullanarak bu seviyede çalışır: gerçek bir HTTP isteği simüle edilir, gerçek (ama izole/geçici) bir veritabanına yazılır, ve yanıtın hem durum kodu hem gövdesi doğrulanır.

3.3. Sözleşme Testleri (Contract Testing)

Mikroservis mimarilerinde (bkz. 22.9), bir API'yi kullanan istemci ile API'nin kendisi genellikle farklı ekipler tarafından, farklı zamanlarda deploy edilir. Sözleşme testi, "API'nin gerçekten OpenAPI şemasında vaat ettiği şekilde davrandığını" otomatik olarak doğrular — API tarafında yapılan bir değişiklik, şemayı (sözleşmeyi) bozuyorsa, bu deploy edilmeden ÖNCE test aşamasında yakalanır.

3.4. Sahte (Mock) API'ler ile Test

22.5'te yazdığımız gibi, BAŞKA bir API'yi TÜKETEN bir istemci kodu test edilirken, her test çalıştığında GERÇEK ağ isteği atmak (1) yavaştır, (2) güvenilmezdir (internet kesintisi testi kırar), (3) dış servise gereksiz yük bindirir. Python'da unittest.mock veya responses/respx kütüphaneleri, gerçek ağ çağrısı yapmadan, "bu URL'ye istek atılırsa şu yanıtı dön" şeklinde SAHTE yanıtlar tanımlamanızı sağlar — testler hem hızlanır hem de dış servisten tamamen bağımsız, güvenilir hale gelir.

SONUÇ

OpenAPI/Swagger standardını, FastAPI'nin otomatik dokümantasyon yeteneklerini, manuel test araçlarını (Postman) ve otomatik test seviyelerini (birim, entegrasyon, sözleşme, mock tabanlı) öğrendik. Artık bir API'yi hem doğru şekilde TANITMAYI hem de GÜVENİLİRLİĞİNİ DOĞRULAMAYI biliyoruz. Sıradaki dosyada (22.8.1), REST'in ötesindeki alternatif API mimarilerini — GraphQL, gRPC ve Webhook'ları — inceleyeceğiz.